■ Snelle Gids voor PyTorch en NumPy

■ NumPy

1. Importeren

```
import numpy as np
```

2. Arrays maken

```
a = np.array([1, 2, 3])
b = np.zeros((2, 3))
c = np.ones((3, 3))
d = np.arange(0, 10, 2)
e = np.linspace(0, 1, 5)
```

3. Eigenschappen

```
print(a.shape)  # Vorm van de array
print(a.dtype)  # Datatype
print(a.ndim)   # Dimensies
```

4. Rekenkundige bewerkingen

```
x = np.array([1, 2, 3])
y = np.array([4, 5, 6])
print(x + y)    # Optellen
print(x * y)    # Vermenigvuldigen
print(np.dot(x, y))  # Inwendig product
```

5. Matrixbewerkingen

```
m = np.array([[1, 2], [3, 4]])
print(np.transpose(m))
print(np.linalg.inv(m))
```

6. Gemiddelde, som en standaardafwijking

```
data = np.array([10, 20, 30])
print(np.mean(data))
print(np.sum(data))
print(np.std(data))
```

■ PyTorch

1. Importeren

```
import torch
```

2. Tensoren maken

```
x = torch.tensor([1, 2, 3])
y = torch.zeros((2, 3))
z = torch.ones((3, 3))
r = torch.rand((2, 2))
```

3. Eigenschappen

```
print(x.shape)
```

```
print(x.dtype)
print(x.device)  # 'cpu' of 'cuda'
```

4. Rekenkundige bewerkingen

```
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
print(a + b)
print(a * b)
print(torch.dot(a, b))
```

5. Matrixbewerkingen

```
m = torch.tensor([[1, 2], [3, 4]], dtype=torch.float32)
print(m.T)            # Transpose
print(torch.inverse(m))
```

6. Gradients en Autograd

```
x = torch.tensor(2.0, requires_grad=True)
y = x ** 3 + 2 * x + 1
y.backward()
print(x.grad)  # Afgeleide dy/dx
```

7. Conversie tussen NumPy en PyTorch

```
import numpy as np
a = np.array([1, 2, 3])
t = torch.from_numpy(a)   # NumPy -> Tensor
n = t.numpy()             # Tensor -> NumPy
```

■ Tip: Gebruik PyTorch voor deep learning en GPU-versnelling; gebruik NumPy voor snelle numerieke berekeningen.